

APLICACIONES WEB

Apéndice — Práctica integradora de Unidad II

CRUD con patrón DAO sin framework en JSP/JSTL/EL, Django y Laravel

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Índice general

A. Práctica integradora: CRUD con patrón DAO sin framework en JSP/JSTL/EL, Django y Laravel	1
Resultado de aprendizaje	1
A.1. Propósito y el sentido de «sin framework»	1
A.2. Modelo de datos común	2
A.3. Pista 1 — Java con JSP, JSTL y EL (JDBC puro, sin Spring)	3
A.3.1. Crear el proyecto en IntelliJ IDEA	3
A.3.2. Estructura final del proyecto	3
A.3.3. Dependencias (pom.xml)	4
A.3.4. La clase de dominio	5
A.3.5. La conexión a la base de datos	5
A.3.6. El DAO con SQL parametrizado	6
A.3.7. El controlador (Servlet)	8
A.3.8. Las vistas (JSP + JSTL + EL)	10
A.3.9. Ejecutar y verificar	11
A.4. Pista 2 — Django con SQL parametrizado (sin ORM)	12
A.4.1. Crear el proyecto en IntelliJ IDEA	12
A.4.2. Estructura final del proyecto	12
A.4.3. Configuración: base de datos y app	13
A.4.4. El dominio	13
A.4.5. El DAO con SQL parametrizado	13
A.4.6. El control (views) y las rutas	14
A.4.7. Las plantillas	16
A.4.8. Ejecutar y verificar	17
A.5. Pista 3 — Laravel con PDO (sin Eloquent)	17
A.5.1. Crear el proyecto en IntelliJ IDEA	18
A.5.2. Estructura final del proyecto	18
A.5.3. La conexión (PDO)	19
A.5.4. El dominio	19
A.5.5. El DAO con SQL parametrizado	19
A.5.6. El control y las rutas	21
A.5.7. Las vistas Blade	23

A.5.8.	Ejecutar y verificar	24
A.6.	Los tres enfoques comparados	24
A.7.	Lista de verificación final	25

Práctica integradora: CRUD con patrón DAO sin framework en JSP/JSTL/EL, Django y Laravel

«El mismo problema, resuelto tres veces, enseña más que tres problemas resueltos una sola vez: lo que permanece es el patrón; lo que cambia es el lenguaje.»

— *Principio pedagógico de la asignatura*

Resultado de aprendizaje

Al concluir esta práctica, el estudiante construye una misma aplicación *CRUD* (crear, leer, actualizar y eliminar) sobre la tabla institucional **estudiantes** en tres pilas tecnológicas del lado del servidor —Java con JSP/JSTL/EL, Django y Laravel—, aplicando en las tres el patrón *Data Access Object* (DAO) y el acceso a datos con SQL parametrizado escrito a mano, sin delegar la persistencia en un ORM. Con ello consolida la separación de responsabilidades entre dominio, acceso a datos, control y presentación.

A.1 Propósito y el sentido de «sin framework»

Esta práctica es el complemento aplicado de la Unidad II. En los capítulos 5 y 6 se presentaron PHP y Java en el servidor, y la persistencia con frameworks que generan el SQL automáticamente. Aquí se hace lo contrario: se escribe el SQL **de forma explícita** y se encapsula en una capa DAO, de modo que el estudiante vea con total claridad qué consulta viaja a la base de datos y cómo se evita la inyección SQL mediante parámetros enlazados.

Concepto — El patrón DAO

Un *Data Access Object* (DAO) es una clase cuya única responsabilidad es traducir entre los objetos del dominio (por ejemplo, **Estudiante**) y las filas de una tabla. Expone métodos de negocio (**listar**, **buscarPorId**, **insertar**, **actualizar**, **eliminar**) y oculta por completo los detalles de JDBC, PDO o el cursor de base de datos. Quien usa el DAO no sabe —ni necesita saber— que por dentro hay SQL.

La expresión «sin framework» se refiere de manera estricta a la **capa de persistencia**: no se utiliza el ORM (Hibernate/JPA en Java, el ORM de Django, ni Eloquent en Laravel). En la pista de Django y en la de Laravel sí se emplea el framework para el enrutamiento y las plantillas —es su razón de ser—, pero el acceso a datos se resuelve con una conexión y consultas propias. Así el mismo patrón DAO se muestra portable entre lenguajes, y se evidencia que el ORM es una comodidad, no una obligación.

Las cuatro capas que se repiten en las tres pistas

1. **Dominio:** la clase **Estudiante** (datos puros).
2. **Acceso a datos:** una clase de *conexión* y un **EstudianteDAO** con SQL parametrizado.
3. **Control:** un servlet/controlador que recibe la petición, decide la acción y elige la vista.
4. **Presentación:** las vistas (JSP, plantilla Django o Blade) que solo muestran datos, sin lógica de negocio.

A.2 Modelo de datos común

Las tres aplicaciones operan sobre la misma base de datos PostgreSQL **appweb2026ppa** y la misma tabla **estudiantes**, ya empleada en la Unidad II. El esquema se crea una sola vez; las tres pistas lo reutilizan sin cambios.

Crear en (cualquier pista): `db/schema.sql`, y ejecutarlo en PostgreSQL.

```

1 CREATE TABLE IF NOT EXISTS estudiantes (
2     id          SERIAL          PRIMARY KEY,
3     nombre     VARCHAR(100) NOT NULL,
4     correo     VARCHAR(120) NOT NULL,
5     carrera    VARCHAR(80)  NOT NULL,
6     creado     TIMESTAMP      NOT NULL DEFAULT NOW()
7 );

```

Listado A.1: Esquema compartido de la tabla estudiantes

Datos de conexión usados en todo el apéndice

Servidor `localhost`, puerto `5432`, base `appweb2026ppa`, usuario `postgres`, contraseña `P@$7Gr3$QL`. En un proyecto real, estas credenciales no se escriben en el código fuente: se leen de variables de entorno o de un archivo de configuración fuera del control de versiones.

Se muestran aquí en el código solo para que la práctica sea reproducible de principio a fin.

A.3 Pista 1 — Java con JSP, JSTL y EL (JDBC puro, sin Spring)

Esta pista usa únicamente la plataforma base de Java para la web: servlets como controladores, JSP con JSTL y EL como vistas, y JDBC para el acceso a datos. El servidor de aplicaciones es Apache Tomcat 11, que implementa Jakarta Servlet 6.1, Jakarta Server Pages 4.0 y Jakarta Expression Language 6.0 [3].

A.3.1 Crear el proyecto en IntelliJ IDEA

1. Menú **File** > **New** > **Project**.
2. En el panel izquierdo, elija **Jakarta EE** (en IntelliJ IDEA Ultimate). Si prefiere partir de cero, elija **Maven Archetype** con `maven-archetype-webapp`.
3. **Name:** `gestion-estudiantes-jsp`. **Build system:** Maven. **JDK:** 21 (LTS).
4. **Template:** *Web application*. **Application server:** agregue *Tomcat 11* con el botón **New...** si aún no está registrado, indicando la carpeta donde descomprimió Tomcat.
5. **Create**. IntelliJ genera el esqueleto Maven con empaquetado **war**.

Lo que IntelliJ genera automáticamente

```

1 gestion-estudiantes-jsp/
2 |-- pom.xml                      (IntelliJ)
3 '-- src/
4     '-- main/
5         |-- java/                 (IntelliJ, vacío)
6         |-- resources/            (IntelliJ, vacío)
7         '-- webapp/
8             |-- WEB-INF/
9                 |-- web.xml        (IntelliJ)
10                '-- index.jsp      (IntelliJ)

```

Listado A.2: Esqueleto inicial creado por IntelliJ IDEA

A.3.2 Estructura final del proyecto

A partir del esqueleto, el programador crea los paquetes y archivos marcados como (*programador*). La organización por capas es la columna vertebral de toda la práctica.

```

1 gestion-estudiantes-jsp/
2 |-- pom.xml                      (programador: dependencias)
3 |-- db/
4 |   '-- schema.sql              (programador)
5 '-- src/main/
6     |-- java/ec/edu/uteq/appweb/

```

```

7      |      |-- modelo/
8      |      |      '-- Estudiante.java                (programador: dominio)
9      |      |-- dao/
10     |      |      |-- ConexionBD.java                (programador: conexion)
11     |      |      '-- EstudianteDAO.java              (programador: acceso a datos)
12     |      '-- controlador/
13     |          '-- EstudianteServlet.java              (programador: control)
14     '-- webapp/
15         |-- WEB-INF/
16         |   |-- web.xml                                (IntelliJ)
17         |   '-- vistas/
18         |       |-- lista.jsp                          (programador: vista)
19         |       '-- formulario.jsp                     (programador: vista)
20     '-- index.jsp                                     (IntelliJ; redirige al servlet)

```

Listado A.3: Estructura completa de la pista JSP/JSTL/EL

A.3.3 Dependencias (pom.xml)

Las API de Servlet y JSP las provee Tomcat en tiempo de ejecución, por eso su scope es provided. JSTL y el driver de PostgreSQL sí se empaquetan dentro del WAR.

Editar: pom.xml

```

1  <properties>
2      <maven.compiler.release>21</maven.compiler.release>
3      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
4  </properties>
5
6  <dependencies>
7      <!-- API de Jakarta Servlet (la provee Tomcat) -->
8      <dependency>
9          <groupId>jakarta.servlet</groupId>
10         <artifactId>jakarta.servlet-api</artifactId>
11         <version>6.1.0</version>
12         <scope>provided</scope>
13     </dependency>
14     <!-- API de Jakarta Server Pages (la provee Tomcat) -->
15     <dependency>
16         <groupId>jakarta.servlet.jsp</groupId>
17         <artifactId>jakarta.servlet.jsp-api</artifactId>
18         <version>4.0.0</version>
19         <scope>provided</scope>
20     </dependency>
21     <!-- JSTL: API + implementacion (se empaquetan en el WAR) -->
22     <dependency>
23         <groupId>jakarta.servlet.jsp.jstl</groupId>
24         <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
25         <version>3.0.2</version>
26     </dependency>
27     <dependency>
28         <groupId>org.glassfish.web</groupId>

```



```

29     <artifactId>jakarta.servlet.jsp.jstl</artifactId>
30     <version>3.0.1</version>
31 </dependency>
32 <!-- Driver JDBC de PostgreSQL -->
33 <dependency>
34     <groupId>org.postgresql</groupId>
35     <artifactId>postgresql</artifactId>
36     <version>42.7.11</version>
37 </dependency>
38 </dependencies>

```

Listado A.4: Dependencias del proyecto JSP

A.3.4 La clase de dominio

Para que EL pueda leer `${e.nombre}` en la JSP, el objeto debe ser un *JavaBean*: atributos privados y métodos `get/set` públicos. Por eso se usa una clase clásica y no un `record`.

Crear en: `src/main/java/ec/edu/uteq/appweb/modelo/Estudiante.java`

```

1 package ec.edu.uteq.appweb.modelo;
2
3 public class Estudiante {
4     private int id;
5     private String nombre;
6     private String correo;
7     private String carrera;
8
9     public Estudiante() { }
10
11     public Estudiante(int id, String nombre, String correo, String carrera) {
12         this.id = id;
13         this.nombre = nombre;
14         this.correo = correo;
15         this.carrera = carrera;
16     }
17     public int getId() { return id; }
18     public void setId(int id) { this.id = id; }
19     public String getNombre() { return nombre; }
20     public void setNombre(String nombre) { this.nombre = nombre; }
21     public String getCorreo() { return correo; }
22     public void setCorreo(String correo) { this.correo = correo; }
23     public String getCarrera() { return carrera; }
24     public void setCarrera(String carrera) { this.carrera = carrera; }
25 }

```

Listado A.5: Estudiante.java (JavaBean del dominio)

A.3.5 La conexión a la base de datos

Desde JDBC 4 el driver se registra solo (vía *ServiceLoader*), de modo que no hace falta `Class.forName`. La conexión se centraliza en una clase con un único método estático.

Crear en: src/main/java/ec/edu/uteq/appweb/dao/ConexionBD.java

```

1 package ec.edu.uteq.appweb.dao;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public final class ConexionBD {
8     private static final String URL =
9         "jdbc:postgresql://localhost:5432/appweb2026ppa";
10    private static final String USUARIO = "postgres";
11    private static final String CLAVE = "P@$7Gr3$QL";
12
13    private ConexionBD() { }
14
15    public static Connection obtener() throws SQLException {
16        return DriverManager.getConnection(URL, USUARIO, CLAVE);
17    }
18 }

```

Listado A.6: ConexionBD.java (conexion JDBC centralizada)

A.3.6 El DAO con SQL parametrizado

Cada método abre su conexión con *try-with-resources*: así la conexión, la sentencia preparada (PreparedStatement) y el conjunto de resultados (ResultSet) se cierran de forma automática. Todos los datos externos viajan como parámetros (?), nunca concatenados: así se previene la inyección SQL.

Crear en: dao/EstudianteDAO.java (parte 1 de 2: lectura)

```

1 package ec.edu.uteq.appweb.dao;
2
3 import ec.edu.uteq.appweb.modelo.Estudiante;
4 import java.sql.Connection;
5 import java.sql.PreparedStatement;
6 import java.sql.ResultSet;
7 import java.sql.SQLException;
8 import java.util.ArrayList;
9 import java.util.List;
10
11 public class EstudianteDAO {
12
13     public List<Estudiante> listar() throws SQLException {
14         String sql = "SELECT id, nombre, correo, carrera "
15             + "FROM estudiantes ORDER BY id";
16         List<Estudiante> lista = new ArrayList<>();
17         try (Connection cn = ConexionBD.obtener();
18             PreparedStatement ps = cn.prepareStatement(sql);
19             ResultSet rs = ps.executeQuery()) {
20             while (rs.next()) {
21                 lista.add(mapear(rs));
22             }
23         }
24     }
25 }

```

```
22     }
23 }
24 return lista;
25 }
26
27 public Estudiante buscarPorId(int id) throws SQLException {
28     String sql = "SELECT id, nombre, correo, carrera "
29         + "FROM estudiantes WHERE id = ?";
30     try (Connection cn = ConexionBD.obtener();
31         PreparedStatement ps = cn.prepareStatement(sql)) {
32         ps.setInt(1, id);
33         try (ResultSet rs = ps.executeQuery()) {
34             return rs.next() ? mapear(rs) : null;
35         }
36     }
37 }
```

Listado A.7: EstudianteDAO.java — parte 1: listar y buscar

Continúa en: dao/EstudianteDAO.java (parte 2 de 2: escritura)

```
1     public void insertar(Estudiante e) throws SQLException {
2         String sql = "INSERT INTO estudiantes (nombre, correo, carrera) "
3             + "VALUES (?, ?, ?)";
4         try (Connection cn = ConexionBD.obtener();
5             PreparedStatement ps = cn.prepareStatement(sql)) {
6             ps.setString(1, e.getNombre());
7             ps.setString(2, e.getCorreo());
8             ps.setString(3, e.getCarrera());
9             ps.executeUpdate();
10        }
11    }
12
13    public void actualizar(Estudiante e) throws SQLException {
14        String sql = "UPDATE estudiantes "
15            + "SET nombre = ?, correo = ?, carrera = ? WHERE id = ?";
16        try (Connection cn = ConexionBD.obtener();
17            PreparedStatement ps = cn.prepareStatement(sql)) {
18            ps.setString(1, e.getNombre());
19            ps.setString(2, e.getCorreo());
20            ps.setString(3, e.getCarrera());
21            ps.setInt(4, e.getId());
22            ps.executeUpdate();
23        }
24    }
25
26    public void eliminar(int id) throws SQLException {
27        String sql = "DELETE FROM estudiantes WHERE id = ?";
28        try (Connection cn = ConexionBD.obtener();
29            PreparedStatement ps = cn.prepareStatement(sql)) {
30            ps.setInt(1, id);
31            ps.executeUpdate();
32        }
33    }
```

```

33     }
34
35     private Estudiante mapear(ResultSet rs) throws SQLException {
36         return new Estudiante(
37             rs.getInt("id"),
38             rs.getString("nombre"),
39             rs.getString("correo"),
40             rs.getString("carrera"));
41     }
42 }

```

Listado A.8: EstudianteDAO.java — parte 2: insertar, actualizar, eliminar

A.3.7 El controlador (Servlet)

Un único servlet atiende la ruta `/estudiantes` y decide la acción según el parámetro `accion`. Las lecturas usan `doGet`; las escrituras (guardar) usan `doPost` y terminan con un *redirect* para evitar reenvíos del formulario.

Crear en: `.../controlador/EstudianteServlet.java`

```

1  package ec.edu.uteq.appweb.controlador;
2
3  import ec.edu.uteq.appweb.dao.EstudianteDAO;
4  import ec.edu.uteq.appweb.modelo.Estudiante;
5  import jakarta.servlet.ServletException;
6  import jakarta.servlet.annotation.WebServlet;
7  import jakarta.servlet.http.HttpServlet;
8  import jakarta.servlet.http.HttpServletRequest;
9  import jakarta.servlet.http.HttpServletResponse;
10 import java.io.IOException;
11 import java.sql.SQLException;
12 import java.util.List;
13
14 @WebServlet("/estudiantes")
15 public class EstudianteServlet extends HttpServlet {
16
17     private final EstudianteDAO dao = new EstudianteDAO();
18
19     @Override
20     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
21         throws ServletException, IOException {
22         String accion = req.getParameter("accion");
23         if (accion == null) { accion = "listar"; }
24         try {
25             switch (accion) {
26                 case "nuevo" -> mostrarFormulario(req, resp, null);
27                 case "editar" -> {
28                     int id = Integer.parseInt(req.getParameter("id"));
29                     mostrarFormulario(req, resp, dao.buscarPorId(id));
30                 }
31                 case "eliminar" -> {

```

```

32         dao.eliminar(Integer.parseInt(req.getParameter("id")));
33         resp.sendRedirect(req.getContextPath() + "/estudiantes");
34     }
35     default -> listar(req, resp);
36 }
37 } catch (SQLException ex) {
38     throw new ServletException("Error de base de datos", ex);
39 }
40 }
41
42 @Override
43 protected void doPost(HttpServletRequest req, HttpServletResponse resp)
44     throws ServletException, IOException {
45     req.setCharacterEncoding("UTF-8");
46     Estudiante e = new Estudiante();
47     String id = req.getParameter("id");
48     e.setNombre(req.getParameter("nombre"));
49     e.setCorreo(req.getParameter("correo"));
50     e.setCarrera(req.getParameter("carrera"));
51     try {
52         if (id == null || id.isBlank()) {
53             dao.insertar(e);
54         } else {
55             e.setId(Integer.parseInt(id));
56             dao.actualizar(e);
57         }
58     } catch (SQLException ex) {
59         throw new ServletException("Error al guardar", ex);
60     }
61     resp.sendRedirect(req.getContextPath() + "/estudiantes");
62 }
63
64 private void listar(HttpServletRequest req, HttpServletResponse resp)
65     throws SQLException, ServletException, IOException {
66     req.setAttribute("lista", dao.listar());
67     req.getRequestDispatcher("/WEB-INF/vistas/lista.jsp")
68         .forward(req, resp);
69 }
70
71 private void mostrarFormulario(HttpServletRequest req,
72     HttpServletResponse resp, Estudiante e)
73     throws ServletException, IOException {
74     req.setAttribute("estudiante", e);
75     req.getRequestDispatcher("/WEB-INF/vistas/formulario.jsp")
76         .forward(req, resp);
77 }
78 }

```

Listado A.9: EstudianteServlet.java (control de la petición)

A.3.8 Las vistas (JSP + JSTL + EL)

Las vistas van en WEB-INF/vistas/ para que no se puedan abrir directamente por URL: solo el servlet las despacha. La directiva `taglib` con URI `jakarta.tags.core` habilita JSTL; el `c:out` escapa la salida (defensa anti-XSS).

Crear en: `src/main/webapp/WEB-INF/vistas/lista.jsp`

```
1 <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
2 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
3 <c:set var="ctx" value="${pageContext.request.contextPath}"/>
4 <!DOCTYPE html>
5 <html lang="es">
6 <head><meta charset="UTF-8"><title>Estudiantes</title></head>
7 <body>
8   <h1>Listado de estudiantes</h1>
9   <p><a href="${ctx}/estudiantes?accion=nuevo">Nuevo estudiante</a></p>
10  <table border="1">
11    <tr><th>ID</th><th>Nombre</th><th>Correo</th>
12      <th>Carrera</th><th>Acciones</th></tr>
13    <c:forEach var="e" items="${lista}">
14      <tr>
15        <td>${e.id}</td>
16        <td><c:out value="${e.nombre}"/></td>
17        <td><c:out value="${e.correo}"/></td>
18        <td><c:out value="${e.carrera}"/></td>
19        <td>
20          <a href="${ctx}/estudiantes?accion=editar&id=${e.id}">Editar</a>
21          <a href="${ctx}/estudiantes?accion=eliminar&id=${e.id}"
22            onclick="return confirm('Eliminar?');">Eliminar</a>
23        </td>
24      </tr>
25    </c:forEach>
26  </table>
27 </body>
28 </html>
```

Listado A.10: lista.jsp (listado con JSTL y EL)

Crear en: `src/main/webapp/WEB-INF/vistas/formulario.jsp`

```
1 <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
2 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
3 <c:set var="ctx" value="${pageContext.request.contextPath}"/>
4 <!DOCTYPE html>
5 <html lang="es">
6 <head><meta charset="UTF-8"><title>Estudiante</title></head>
7 <body>
8   <h1>
9     <c:choose>
10       <c:when test="${estudiante != null}">Editar</c:when>
11       <c:otherwise>Nuevo</c:otherwise>
12     </c:choose> estudiante
```

```

13 </h1>
14 <form method="post" action="${ctx}/estudiantes">
15   <input type="hidden" name="id" value="${estudiante.id}">
16   <p>Nombre:
17     <input type="text" name="nombre"
18       value="<c:out value='${estudiante.nombre}'/>" required></p>
19   <p>Correo:
20     <input type="email" name="correo"
21       value="<c:out value='${estudiante.correo}'/>" required></p>
22   <p>Carrera:
23     <input type="text" name="carrera"
24       value="<c:out value='${estudiante.carrera}'/>" required></p>
25   <button type="submit">Guardar</button>
26   <a href="${ctx}/estudiantes">Cancelar</a>
27 </form>
28 </body>
29 </html>

```

Listado A.11: formulario.jsp (alta y edicion)

Para que la raíz del sitio lleve al listado, `index.jsp` solo redirige:

Editar: `src/main/webapp/index.jsp`

```

1 <% response.sendRedirect(request.getContextPath() + "/estudiantes"); %>

```

Listado A.12: index.jsp (redireccion al servlet)

A.3.9 Ejecutar y verificar

1. Ejecute `db/schema.sql` en PostgreSQL (pestaña *Database* de IntelliJ o `psql`).
2. Run > Edit Configurations > + > Tomcat Server > Local. En la pestaña *Deployment* agregue el artefacto `gestion-estudiantes-jsp:war exploded`.
3. Pulse Run. IntelliJ despliega en Tomcat y abre el navegador en la ruta de contexto `/gestion-estudiantes-jsp/` (puerto 8080 por defecto).
4. Verifique el ciclo completo: crear, listar, editar y eliminar un estudiante. En la consola no deben aparecer trazas de error.

Errores típicos en la pista JSP

- **No suitable driver:** falta la dependencia del driver PostgreSQL en `pom.xml` o no se recargó Maven.
- Etiquetas `<c:forEach>` que aparecen como texto: el URI del taglib no es `jakarta.tags.core`, o falta la implementación de JSTL (segunda dependencia).
- 404 al abrir la JSP directamente: es lo esperado; las vistas están en `WEB-INF` y solo se acceden vía el servlet.

A.4 Pista 2 — Django con SQL parametrizado (sin ORM)

Django aporta el enrutamiento, las plantillas y la gestión de la petición; el acceso a datos, en cambio, se resuelve con un DAO que ejecuta SQL a mano mediante el cursor de `django.db.connection`. No se definen modelos del ORM para `estudiantes`. Se usa Django 5.2 LTS sobre Python 3.13 [1].

A.4.1 Crear el proyecto en IntelliJ IDEA

1. Active el plugin *Python* (**Settings** > **Plugins**) en IntelliJ IDEA Ultimate.
2. **File** > **New** > **Project** > **Python**. Defina un entorno virtual (*Virtualenv*) con el intérprete Python 3.13.
3. Abra la *Terminal* integrada e instale las dependencias:

```
1 pip install "Django>=5.2,<6.0" "psycopg[binary]"
2 django-admin startproject gestion .
3 python manage.py startapp estudiantes
```

Listado A.13: Instalación de Django y el adaptador de PostgreSQL

Lo que generan `startproject` y `startapp`

```
1 gestion/                                (raíz del proyecto)
2 |-- manage.py                          (django-admin)
3 |-- gestion/                          (django-admin: configuracion)
4 |   |-- __init__.py
5 |   |-- settings.py
6 |   |-- urls.py
7 |   |-- asgi.py
8 |   '-- wsgi.py
9 '-- estudiantes/                      (django-admin: la app)
10    |-- __init__.py
11    |-- admin.py
12    |-- apps.py
13    |-- migrations/
14    |-- models.py                      (NO se usa: no hay ORM)
15    |-- tests.py
16    '-- views.py
```

Listado A.14: Esqueleto creado por `django-admin`

A.4.2 Estructura final del proyecto

```
1 gestion/
2 |-- manage.py                          (django-admin)
3 |-- db/schema.sql                      (programador)
4 |-- gestion/
5 |   |-- settings.py                    (programador: BD + app)
6 |   '-- urls.py                       (programador: incluye la app)
```



```

7  '-- estudiantes/
8      |-- dominio.py                (programador: dominio)
9      |-- dao.py                   (programador: acceso a datos)
10     |-- views.py                 (programador: control)
11     |-- urls.py                  (programador: rutas de la app)
12     '-- templates/estudiantes/
13         |-- lista.html           (programador: vista)
14         '-- formulario.html      (programador: vista)

```

Listado A.15: Estructura completa de la pista Django

A.4.3 Configuración: base de datos y app

Editar: `gestion/settings.py`

```

1  INSTALLED_APPS = [
2      "django.contrib.staticfiles",
3      "estudiantes",
4  ]
5
6  DATABASES = {
7      "default": {
8          "ENGINE": "django.db.backends.postgresql",
9          "NAME": "appweb2026ppa",
10         "USER": "postgres",
11         "PASSWORD": "P@$7Gr3$QL",
12         "HOST": "localhost",
13         "PORT": "5432",
14     }
15 }

```

Listado A.16: `settings.py` (conexión PostgreSQL y registro de la app)

A.4.4 El dominio

Crear en: `estudiantes/dominio.py`

```

1  from dataclasses import dataclass
2
3
4  @dataclass
5  class Estudiante:
6      id: int | None
7      nombre: str
8      correo: str
9      carrera: str

```

Listado A.17: `dominio.py` (la clase `Estudiante`)

A.4.5 El DAO con SQL parametrizado

El cursor de Django usa marcadores `%s` (estilo DB-API). Al pasar los valores como segundo argumento de `execute`, la biblioteca los enlaza de forma segura: no hay concatenación, no hay

inyeccion SQL.

Crear en: estudiantes/dao.py

```
1 from django.db import connection
2 from .dominio import Estudiante
3
4
5 class EstudianteDAO:
6     """Acceso a datos con SQL parametrizado, sin el ORM de Django."""
7
8     def listar(self):
9         sql = ("SELECT id, nombre, correo, carrera "
10              "FROM estudiantes ORDER BY id")
11         with connection.cursor() as cur:
12             cur.execute(sql)
13             return [self._mapear(fila) for fila in cur.fetchall()]
14
15     def buscar_por_id(self, id_est):
16         sql = ("SELECT id, nombre, correo, carrera "
17              "FROM estudiantes WHERE id = %s")
18         with connection.cursor() as cur:
19             cur.execute(sql, [id_est])
20             fila = cur.fetchone()
21             return self._mapear(fila) if fila else None
22
23     def insertar(self, e):
24         sql = ("INSERT INTO estudiantes (nombre, correo, carrera) "
25              "VALUES (%s, %s, %s)")
26         with connection.cursor() as cur:
27             cur.execute(sql, [e.nombre, e.correo, e.carrera])
28
29     def actualizar(self, e):
30         sql = ("UPDATE estudiantes "
31              "SET nombre = %s, correo = %s, carrera = %s WHERE id = %s")
32         with connection.cursor() as cur:
33             cur.execute(sql, [e.nombre, e.correo, e.carrera, e.id])
34
35     def eliminar(self, id_est):
36         sql = "DELETE FROM estudiantes WHERE id = %s"
37         with connection.cursor() as cur:
38             cur.execute(sql, [id_est])
39
40     @staticmethod
41     def _mapear(fila):
42         return Estudiante(id=fila[0], nombre=fila[1],
43                           correo=fila[2], carrera=fila[3])
```

Listado A.18: dao.py (CRUD con cursor, sin el ORM)

A.4.6 El control (views) y las rutas

Crear en: estudiantes/views.py

```
1 from django.shortcuts import render, redirect
2 from .dao import EstudianteDAO
3 from .dominio import Estudiante
4
5 dao = EstudianteDAO()
6
7
8 def listar(request):
9     return render(request, "estudiantes/lista.html",
10                   {"estudiantes": dao.listar()})
11
12
13 def nuevo(request):
14     return render(request, "estudiantes/formulario.html",
15                   {"estudiante": None})
16
17
18 def editar(request, id_est):
19     return render(request, "estudiantes/formulario.html",
20                   {"estudiante": dao.buscar_por_id(id_est)})
21
22
23 def guardar(request):
24     id_txt = request.POST.get("id")
25     e = Estudiante(
26         id=int(id_txt) if id_txt else None,
27         nombre=request.POST.get("nombre"),
28         correo=request.POST.get("correo"),
29         carrera=request.POST.get("carrera"))
30     if e.id:
31         dao.actualizar(e)
32     else:
33         dao.insertar(e)
34     return redirect("estudiantes:listar")
35
36
37 def eliminar(request, id_est):
38     dao.eliminar(id_est)
39     return redirect("estudiantes:listar")
```

Listado A.19: views.py (control de la petición)

Crear en: estudiantes/urls.py

```
1 from django.urls import path
2 from . import views
3
4 app_name = "estudiantes"
5
6 urlpatterns = [
7     path("", views.listar, name="listar"),
8     path("nuevo/", views.nuevo, name="nuevo"),
```

```
9     path("editar/<int:id_est>/", views.editar, name="editar"),
10    path("guardar/", views.guardar, name="guardar"),
11    path("eliminar/<int:id_est>/", views.eliminar, name="eliminar"),
12 ]
```

Listado A.20: urls.py de la app

Editar: gestion/urls.py

```
1 from django.urls import path, include
2
3 urlpatterns = [
4     path("estudiantes/", include("estudiantes.urls")),
5 ]
```

Listado A.21: urls.py del proyecto (incluye la app)

A.4.7 Las plantillas

El formulario incluye {% csrf_token%}: Django rechaza los POST sin ese testigo, lo que protege contra CSRF.

Crear en: estudiantes/templates/estudiantes/lista.html

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head><meta charset="UTF-8"><title>Estudiantes</title></head>
4 <body>
5     <h1>Listado de estudiantes</h1>
6     <p><a href="{% url 'estudiantes:nuevo' %}">Nuevo estudiante</a></p>
7     <table border="1">
8         <tr><th>ID</th><th>Nombre</th><th>Correo</th>
9             <th>Carrera</th><th>Acciones</th></tr>
10        {% for e in estudiantes %}
11            <tr>
12                <td>{{ e.id }}</td>
13                <td>{{ e.nombre }}</td>
14                <td>{{ e.correo }}</td>
15                <td>{{ e.carrera }}</td>
16                <td>
17                    <a href="{% url 'estudiantes:editar' e.id %}">Editar</a>
18                    <a href="{% url 'estudiantes:eliminar' e.id %}"
19                        onclick="return confirm('Eliminar?');">Eliminar</a>
20                </td>
21            </tr>
22        {% endfor %}
23    </table>
24 </body>
25 </html>
```

Listado A.22: lista.html (plantilla de listado)

Crear en: estudiantes/templates/estudiantes/formulario.html

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head><meta charset="UTF-8"><title>Estudiante</title></head>
4 <body>
5   <h1>{% if estudiante %}Editar{% else %}Nuevo{% endif %} estudiante</h1>
6   <form method="post" action="{% url 'estudiantes:guardar' %}">
7     {% csrf_token %}
8     <input type="hidden" name="id"
9       value="{% if estudiante %}{{ estudiante.id }}{% endif %}">
10    <p>Nombre: <input type="text" name="nombre"
11      value="{% if estudiante %}{{ estudiante.nombre }}{% endif %}"
12      required></p>
13    <p>Correo: <input type="email" name="correo"
14      value="{% if estudiante %}{{ estudiante.correo }}{% endif %}"
15      required></p>
16    <p>Carrera: <input type="text" name="carrera"
17      value="{% if estudiante %}{{ estudiante.carrera }}{% endif %}"
18      required></p>
19    <button type="submit">Guardar</button>
20    <a href="{% url 'estudiantes:listar' %}">Cancelar</a>
  </form>
</body>
</html>

```

Listado A.23: formulario.html (alta y edicion)

A.4.8 Ejecutar y verificar

1. Ejecute db/schema.sql en PostgreSQL.
2. En la Terminal: `python manage.py runserver`.
3. Abra `http://localhost:8000/estudiantes/` y pruebe crear, listar, editar y eliminar.

Errores tipicos en la pista Django

- `ModuleNotFoundError: psycopg`: falta instalar `psycopg[binary]` en el entorno virtual activo.
- `TemplateDoesNotExist:` la plantilla no esta en `estudiantes/templates/estudiantes/` o falta registrar la app en `INSTALLED_APPS`.
- `403 (CSRF verification failed):` el formulario no incluye `{% csrf_token %}`.

A.5 Pista 3 — Laravel con PDO (sin Eloquent)

Laravel aporta el enrutamiento, los controladores y las plantillas Blade; el acceso a datos se resuelve con un DAO que usa PDO directamente, sin Eloquent. Se usa Laravel 13 sobre PHP 8.4 [2, 4].

A.5.1 Crear el proyecto en IntelliJ IDEA

1. Active el plugin *PHP* en IntelliJ IDEA Ultimate y configure el interprete PHP 8.4 (**Settings** > **PHP**). Asegure la extension `pdo_pgsql` habilitada.
2. Cree el proyecto con Composer desde la Terminal:

```
1 composer create-project laravel/laravel gestion-estudiantes
2 cd gestion-estudiantes
3 php artisan key:generate
```

Listado A.24: Creacion del proyecto Laravel

Lo que genera Composer (estructura esencial)

```
1 gestion-estudiantes/
2 |-- app/
3 |   |-- Http/Controllers/Controller.php      (Composer)
4 |   |-- Models/                               (Composer; no se usa
5 |       Eloquent)
6 |   '-- Providers/
7 |-- bootstrap/ config/ database/
8 |-- public/index.php                         (Composer: punto de
9 |     entrada)
10 |-- resources/views/                        (Composer)
11 |-- routes/web.php                          (Composer)
12 |-- .env                                    (Composer: configuracion)
13 |-- composer.json
14 |-- artisan
```

Listado A.25: Esqueleto creado por laravel/laravel

A.5.2 Estructura final del proyecto

Las clases del DAO y del dominio se ubican en carpetas nuevas dentro de `app/`, aprovechando el autocargado PSR-4 (`App\`).

```
1 gestion-estudiantes/
2 |-- db/schema.sql                            (programador)
3 |-- app/
4 |   |-- Dominio/Estudiante.php              (programador: dominio)
5 |   |-- Dao/
6 |       |-- ConexionBD.php                  (programador: conexion PDO)
7 |       '-- EstudianteDao.php               (programador: acceso a datos)
8 |   '-- Http/Controllers/
9 |       '-- EstudianteController.php        (programador: control)
10 |-- routes/web.php                          (programador: rutas)
11 '-- resources/views/estudiantes/
12     |-- lista.blade.php                     (programador: vista)
13     '-- formulario.blade.php                (programador: vista)
```

Listado A.26: Estructura completa de la pista Laravel

A.5.3 La conexion (PDO)

Crear en: app/Dao/ConexionBD.php

```

1 <?php
2
3 namespace App\Dao;
4
5 use PDO;
6
7 class ConexionBD
8 {
9     private static ?PDO $pdo = null;
10
11     public static function obtener(): PDO
12     {
13         if (self::$pdo === null) {
14             $dsn = 'pgsql:host=localhost;port=5432;dbname=appweb2026ppa';
15             self::$pdo = new PDO($dsn, 'postgres', 'P@$7Gr3$QL', [
16                 PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
17                 PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
18             ]);
19         }
20         return self::$pdo;
21     }
22 }

```

Listado A.27: ConexionBD.php (conexion PDO reutilizable)

A.5.4 El dominio

Crear en: app/Dominio/Estudiante.php

```

1 <?php
2
3 namespace App\Dominio;
4
5 class Estudiante
6 {
7     public function __construct(
8         public ?int $id,
9         public string $nombre,
10        public string $correo,
11        public string $carrera
12    ) {}
13 }

```

Listado A.28: Estudiante.php (dominio con propiedades promovidas)

A.5.5 El DAO con SQL parametrizado

PDO usa marcadores con nombre (:nombre). El metodo `execute` recibe el arreglo de valores y los enlaza de forma segura.

Crear en: app/Dao/EstudianteDao.php

```
1 <?php
2
3 namespace App\Dao;
4
5 use App\Dominio\Estudiante;
6
7 class EstudianteDao
8 {
9     public function listar(): array
10     {
11         $sql = 'SELECT id, nombre, correo, carrera '
12             . 'FROM estudiantes ORDER BY id';
13         $stmt = ConexionBD::obtener()->query($sql);
14         return array_map([$this, 'mapear'], $stmt->fetchAll());
15     }
16
17     public function buscarPorId(int $id): ?Estudiante
18     {
19         $sql = 'SELECT id, nombre, correo, carrera '
20             . 'FROM estudiantes WHERE id = :id';
21         $stmt = ConexionBD::obtener()->prepare($sql);
22         $stmt->execute([':id' => $id]);
23         $fila = $stmt->fetch();
24         return $fila ? $this->mapear($fila) : null;
25     }
26
27     public function insertar(Estudiante $e): void
28     {
29         $sql = 'INSERT INTO estudiantes (nombre, correo, carrera) '
30             . 'VALUES (:nombre, :correo, :carrera)';
31         $stmt = ConexionBD::obtener()->prepare($sql);
32         $stmt->execute([
33             ':nombre' => $e->nombre,
34             ':correo' => $e->correo,
35             ':carrera' => $e->carrera,
36         ]);
37     }
38
39     public function actualizar(Estudiante $e): void
40     {
41         $sql = 'UPDATE estudiantes SET nombre = :nombre, '
42             . 'correo = :correo, carrera = :carrera WHERE id = :id';
43         $stmt = ConexionBD::obtener()->prepare($sql);
44         $stmt->execute([
45             ':nombre' => $e->nombre,
46             ':correo' => $e->correo,
47             ':carrera' => $e->carrera,
48             ':id' => $e->id,
49         ]);
50     }
51 }
```



```
52     public function eliminar(int $id): void
53     {
54         $sql = 'DELETE FROM estudiantes WHERE id = :id';
55         $stmt = ConexionBD::obtener()->prepare($sql);
56         $stmt->execute([':id' => $id]);
57     }
58
59     private function mapear(array $fila): Estudiante
60     {
61         return new Estudiante(
62             (int) $fila['id'],
63             $fila['nombre'],
64             $fila['correo'],
65             $fila['carrera']
66         );
67     }
68 }
```

Listado A.29: EstudianteDao.php (CRUD con PDO, sin Eloquent)

A.5.6 El control y las rutas

Crear en: app/Http/Controllers/EstudianteController.php

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Dao\EstudianteDao;
6  use App\Dominio\Estudiante;
7  use Illuminate\Http\Request;
8
9  class EstudianteController extends Controller
10 {
11     private EstudianteDao $dao;
12
13     public function __construct()
14     {
15         $this->dao = new EstudianteDao();
16     }
17
18     public function index()
19     {
20         return view('estudiantes.lista',
21             ['estudiantes' => $this->dao->listar()]);
22     }
23
24     public function nuevo()
25     {
26         return view('estudiantes.formulario', ['estudiante' => null]);
27     }
28
29     public function editar(int $id)
```

```
30 {
31     return view('estudiantes.formulario',
32         ['estudiante' => $this->dao->buscarPorId($id)]);
33 }
34
35 public function guardar(Request $request)
36 {
37     $datos = $request->validate([
38         'id' => 'nullable|integer',
39         'nombre' => 'required|string|max:100',
40         'correo' => 'required|email|max:120',
41         'carrera' => 'required|string|max:80',
42     ]);
43     $e = new Estudiante(
44         $datos['id'] ?? null,
45         $datos['nombre'], $datos['correo'], $datos['carrera']);
46     $e->id ? $this->dao->actualizar($e) : $this->dao->insertar($e);
47     return redirect()->route('estudiantes.index');
48 }
49
50 public function eliminar(int $id)
51 {
52     $this->dao->eliminar($id);
53     return redirect()->route('estudiantes.index');
54 }
55 }
```

Listado A.30: EstudianteController.php (control de la petición)

Editar: routes/web.php

```
1 <?php
2
3 use App\Http\Controllers\EstudianteController;
4 use Illuminate\Support\Facades\Route;
5
6 Route::get('/estudiantes', [EstudianteController::class, 'index'])
7     ->name('estudiantes.index');
8 Route::get('/estudiantes/nuevo', [EstudianteController::class, 'nuevo'])
9     ->name('estudiantes.nuevo');
10 Route::get('/estudiantes/{id}/editar', [EstudianteController::class,
11     'editar'])
12     ->name('estudiantes.editar');
13 Route::post('/estudiantes/guardar', [EstudianteController::class, 'guardar'])
14     ->name('estudiantes.guardar');
15 Route::post('/estudiantes/{id}/eliminar', [EstudianteController::class,
16     'eliminar'])
17     ->name('estudiantes.eliminar');
```

Listado A.31: web.php (rutas del CRUD)

A.5.7 Las vistas Blade

La directiva `@csrf` inserta el testigo anti-CSRF; las llaves dobles `{{ }}` escapan la salida (anti-XSS).

Crear en: `resources/views/estudiantes/lista.blade.php`

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head><meta charset="UTF-8"><title>Estudiantes</title></head>
4 <body>
5   <h1>Listado de estudiantes</h1>
6   <p><a href="{{ route('estudiantes.nuevo') }}">Nuevo estudiante</a></p>
7   <table border="1">
8     <tr><th>ID</th><th>Nombre</th><th>Correo</th>
9       <th>Carrera</th><th>Acciones</th></tr>
10    @foreach ($estudiantes as $e)
11      <tr>
12        <td>{{ $e->id }}</td>
13        <td>{{ $e->nombre }}</td>
14        <td>{{ $e->correo }}</td>
15        <td>{{ $e->carrera }}</td>
16        <td>
17          <a href="{{ route('estudiantes.editar', $e->id) }}">Editar</a>
18          <form method="post" style="display:inline"
19            action="{{ route('estudiantes.eliminar', $e->id) }}"
20            onsubmit="return confirm('Eliminar?');">
21            @csrf
22            <button type="submit">Eliminar</button>
23          </form>
24        </td>
25      </tr>
26    @endforeach
27  </table>
28 </body>
29 </html>

```

Listado A.32: `lista.blade.php` (listado)

Crear en: `resources/views/estudiantes/formulario.blade.php`

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head><meta charset="UTF-8"><title>Estudiante</title></head>
4 <body>
5   <h1>{{ $estudiante ? 'Editar' : 'Nuevo' }} estudiante</h1>
6   <form method="post" action="{{ route('estudiantes.guardar') }}">
7     @csrf
8     <input type="hidden" name="id" value="{{ $estudiante->id ?? '' }}">
9     <p>Nombre: <input type="text" name="nombre"
10       value="{{ $estudiante->nombre ?? '' }}" required></p>
11     <p>Correo: <input type="email" name="correo"
12       value="{{ $estudiante->correo ?? '' }}" required></p>
13     <p>Carrera: <input type="text" name="carrera"

```

```

14         value="{{ $estudiante->carrera ?? '' }}" required></p>
15         <button type="submit">Guardar</button>
16         <a href="{{ route('estudiantes.index') }}">Cancelar</a>
17     </form>
18 </body>
19 </html>

```

Listado A.33: formulario.blade.php (alta y edicion)

A.5.8 Ejecutar y verificar

1. Ejecute db/schema.sql en PostgreSQL.
2. En la Terminal: `php artisan serve`.
3. Abra `http://localhost:8000/estudiantes` y pruebe el ciclo CRUD completo.

Errores tipicos en la pista Laravel

- `could not find driver`: falta habilitar la extension `pdo_pgsql` en `php.ini`.
- `419 (Page Expired)`: el formulario no incluye `@csrf`.
- `Class not found`: el espacio de nombres del archivo no coincide con su carpeta (PSR-4); revise `namespace` y la ruta.

A.6 Los tres enfoques comparados

Las tres pistas resuelven el mismo problema con la misma arquitectura en cuatro capas; lo que cambia es la sintaxis y las piezas que aporta cada plataforma. La tabla A.1 resume las correspondencias.

Tabla A.1: Correspondencia entre las tres pistas del CRUD con DAO

Aspecto	JSP/JSTL/EL	Django	Laravel
Lenguaje / versión	Java 21	Python 3.13	PHP 8.4
Servidor de ejecución	Tomcat 11	runserver (dev)	artisan serve (dev)
Conexión a la BD	JDBC DriverManager	cursor de connection	PDO
Marcador de parámetro	?	%s	:nombre
Control	Servlet	función <i>view</i>	Controller
Presentación	JSP + JSTL + EL	plantilla Django	Blade
Anti-XSS en la vista	<code>c:out</code>	<code>{{ }}</code>	<code>{{ }}</code>
Anti-CSRF	(manual)	<code>csrf_token</code>	<code>@csrf</code>

Lo que el estudiante debe llevarse de esta práctica

1. El patrón DAO es independiente del lenguaje: una clase de conexión y un DAO con métodos de negocio resuelven la persistencia en cualquier pila.
2. El SQL parametrizado (?, %s o :nombre) es la defensa primaria contra la inyección SQL en las tres pilas.
3. El controlador nunca contiene SQL ni la vista contiene lógica de negocio: esa separación es lo que hace mantenible la aplicación.
4. El ORM ahorra código, pero comprender el SQL subyacente —como en este apéndice— es lo que permite depurar y optimizar cuando el ORM no basta.

A.7 Lista de verificación final

1. La tabla `estudiantes` existe en `appweb2026ppa`.
2. Las cuatro capas (dominio, DAO, control, vista) están separadas en archivos distintos.
3. Todas las consultas usan parámetros enlazados; no hay concatenación de cadenas en el SQL.
4. Las vistas escapan la salida del usuario.
5. El ciclo CRUD completo (crear, listar, editar, eliminar) funciona de extremo a extremo en las tres pistas.

Bibliografía

- [1] Django Software Foundation. Django Documentation, Version 5.2 LTS. <https://docs.djangoproject.com/en/5.2/>, 2026. Versión LTS con soporte hasta abril de 2028. Consultado en junio de 2026.
- [2] Laravel LLC. Laravel 13.x Documentation. <https://laravel.com/docs/13.x>, 2026. Lanzada el 17 de marzo de 2026; requiere PHP 8.3 o superior. Consultado en junio de 2026.
- [3] PostgreSQL Global Development Group. PostgreSQL JDBC Driver (pgJDBC) Documentation. <https://jdbc.postgresql.org/documentation/>, 2026. Versión 42.7.11. Consultado en junio de 2026.
- [4] The PHP Documentation Group. PHP Manual: PDO — PHP Data Objects. <https://www.php.net/manual/en/book.pdo.php>, 2026. Consultado en junio de 2026.